

CUSTOMER FEATURE GUIDE

Atlas

The central storage control plane for the Zyvor
product suite.

One intent-based API that maps every product's storage request to Ceph, NFS, ZFS, or the cloud — while owning inventory, ownership, protection, and audit.

by ZyvorAI Labs

What is Atlas?

Atlas lets products ask for intent — "give me production block storage" — instead of wrestling with pool internals, then maps that intent to a real backend through pluggable drivers. It gives you block, file, and object storage from a single gateway, with async provisioning, snapshots, backups, replication, per-tenant governance, and a live console on top. Ceph is the first driver (RBD, CephFS, RGW/S3), with NFS and ZFS backends alongside it and DataBridge adding cloud-to-edge database and object mobility.

3

Storage backends
(Ceph · NFS · ZFS)

6

Database engines
migratable via
DataBridge

80+

REST endpoints across
the control plane

3

Access surfaces —
REST · gRPC · SSE

What's inside this guide

Getting started — access & first workflows	start here
1 · Control Plane & Architecture	6 features
2 · Block, File & Object Storage	6 features
3 · Data Protection	6 features
4 · Multi-Backend & Ceph Operations	6 features
5 · Observability & Metrics	6 features
6 · Alerting & Day-2 Ops	6 features
7 · Governance & Multi-Tenancy	6 features
8 · DataBridge — Database Mobility	6 features
9 · DataBridge — Object Migration	5 features
10 · Disaster Recovery	3 features
11 · Storage Center Console	5 features
Support & contact	→

Access & first workflows

How to reach Atlas and get your first result. Assumes it has been deployed for you.

How to access it

WEB	"Storage Center" React console served from the gateway binary at <code>http://127.0.0.1:5110/</code> (local <code>make run</code>), or <code>http://<node>:30511/</code> on a Ceph-mode NodePort deploy.
CLI	<code>atlasctl</code> headless REST client — <code>cargo run -p atlas-cli -- <cmd></code> (health, ready, discover, pools, volumes, snapshots, backups, buckets, tenants, tokens, ceph-status ...); global flags <code>--base-url</code> (<code>ATLAS_BASE_URL</code>) and <code>--token</code> (<code>ATLAS_TOKEN</code>).
API	REST base <code>/api/atlas/v1/...</code> (all JSON); a <code>tonic</code> gRPC edge (<code>atlas.v1.AtlasStorage</code>) on <code>ATLAS_GRPC_ADDR</code> (<code>:5111</code> , NodePort 30512) with server-streaming <code>WatchJob</code> ; async writes return <code>202 + job_id</code> , streamable over SSE at <code>/api/atlas/v1/jobs/{id}/watch</code> .
LOGIN	Auth is off by default (dev; actor is <code>anonymous</code>). Set <code>ATLAS_AUTH_REQUIRED=1</code> to require an HS256 JWT (<code>Authorization: Bearer <jwt></code>); mint one via <code>POST /api/atlas/v1/auth/tokens</code> .
NEEDS	A Ceph (RBD/CephFS/RGW), NFS, or ZFS backend. For evaluation you need no cluster at all — run the fake driver with <code>ATLAS_CEPH_DRIVER_MODE=fake</code> (this is what <code>make run</code> does).

Your first workflows



Run the gateway fake-first (no cluster)

1. `make run` — starts the gateway with the fake Ceph driver on `127.0.0.1:5110` , runs SQLite migrations, and does one discovery pass.
2. `curl -s localhost:5110/health` returns `{ "status": "ok" }` .
3. Open `http://127.0.0.1:5110/` for the Storage Center console (inventory, capacity, jobs, alerts, tenants).



Populate & query inventory

1. `curl -s -X POST localhost:5110/api/atlas/v1/backends/bkd_ceph_lab/discover` (or `atlasctl discover`) normalizes backend state into the SQLite inventory.
2. `curl -s localhost:5110/api/atlas/v1/pools | jq` and `.../volumes` — expected fake inventory: 3 pools, 2 volumes, 6 OSDs.
3. In the console: Storage Center → Inventory / Pools / Volumes.



Register a live NFS or ZFS backend

1. `POST /api/atlas/v1/backends` with `{ "name": "nas-1", "backend_type": "nfs", ... }` — Atlas instantiates the driver and discovers it immediately (not just a catalog row).
2. Confirm with `GET /api/atlas/v1/backends` ; filter pools by `GET /api/atlas/v1/pools?backend=<id>` .

Provision a volume by intent

1. `POST /api/atlas/v1/volumes` with `{ "tenant_id": "...", "name": "billing-db-root", "size_bytes": 3221225472, "kind": "block", "policy": "database", "owner": {...} }` → `202 + job_id`.
2. Poll `GET /api/atlas/v1/jobs/{id}` (or watch SSE at `/api/atlas/v1/jobs/{id}/watch`) until succeeded.
3. CLI equivalent: `atlasctl create-volume billing-db-root --size-gib 3 --policy database --namespace default`.

Protect a volume (snapshot → backup → schedule)

1. Snapshot now: `POST /api/atlas/v1/volumes/{id}/snapshots`.
2. Provision a bucket then back up off-cluster: `POST /api/atlas/v1/buckets` → `POST /api/atlas/v1/backup-jobs` with `{ "volume_id": "...", "bucket_id": "...", "mode": "data", "keep": 7 }`.
3. Automate it: `POST /api/atlas/v1/volumes/{id}/schedule` with `{ "interval_secs": 3600, "keep": 24 }` (or `kind: "backup"`).

DataBridge migration, fake-first

1. `make run-databridge` runs the whole discover-to-cutover pipeline with no cloud or Kubernetes creds.
2. Or by API: `POST /databridge/sources` → `.../discover` → `POST /databridge/plans` → `.../assess` → `.../provision` → `.../full-load` → `.../cdc/start` → `.../validate` → `.../cutover` (admin, guarded).

Control Plane & Architecture

An intent-driven gateway that decouples every product from the storage underneath it.



Intent-Based Provisioning

Products request an intent class ("production block storage") and Atlas resolves it to a concrete backend, pool, and placement.

Callers stay decoupled from Ceph or any future backend — no pool internals leak into product code.

How · REST `POST /api/atlas/v1/volumes` with a `policy` intent (e.g. "database"), which `atlas-policy` resolves to a `StorageClass`; browse the catalog at `GET /api/atlas/v1/policies`. Console: Storage Center → Volumes → Create.



Pluggable Storage Drivers

A single `StorageDriver` trait fronts every backend, with real Ceph, NFS, ZFS, and Kubernetes drivers plus a fake driver for local runs.

Add a backend once and every product gets it through the same stable API.

How · The driver is chosen by `backend_type` at registration — `POST /api/atlas/v1/backends` { `"backend_type": "ceph|nfs|zfs|kubernetes"` }; run local with `ATLAS_CEPH_DRIVER_MODE=fake`. Console: Storage Center → Backends.



Atlas Gateway

An axum server that centralizes auth, audit, and the API surface, and embeds the Storage Center console in the binary.

One deployable front door for storage across all nine Zyor products.

How · Start it with `make run`; it serves the console at `http://127.0.0.1:5110/` and REST at `/api/atlas/v1/*` from one binary. Health: `curl localhost:5110/health`.



REST + gRPC Surfaces

The same control plane is reachable over REST and a tonic gRPC edge with streaming `WatchJob` for product integrations.

Human tools use REST; product runtimes get typed, streaming gRPC.

How · REST at `/api/atlas/v1/*`; gRPC `atlas.v1.AtlasStorage` on `:5111` — `grpcurl -plaintext <host>:5111 list`, `grpcurl -plaintext <host>:5111 atlas.v1.AtlasStorage/ListPools`.



Inventory & Discovery

A discovery worker normalizes backend state into a SQLite inventory of pools, volumes, and ownership bindings.

A consistent, queryable picture of every asset regardless of which backend holds it.

How · REST `POST /api/atlas/v1/backends/{id}/discover` (or `atlasctl discover`), then read `GET /api/atlas/v1/pools` · `/volumes` · `/clusters`. Console: Storage Center → Inventory.



Durable, Self-Healing State

Restarts recover the job engine (interrupted jobs fail safe, queued jobs re-enqueue), with graceful `SIGTERM` drain and deep readiness probes.

The control plane stays trustworthy across pod restarts and rollouts.

How · Automatic on boot; verify with deep readiness `curl localhost:5110/readyz` (`atlasctl ready`) vs liveness `GET /livez`. Wire k8s `readinessProbe→ /readyz`, `livenessProbe→ /livez`.

Core principle: products request **intent**, not pool internals. Atlas maps intent to a backend and owns inventory, ownership, and audit.

SECTION 2

Block, File & Object Storage

Provision and manage RBD block, CephFS file, and RGW/S3 object storage from one API.



Block Volumes

Create, get, expand, and delete Ceph-backed block volumes (PVCs) as async jobs that return a job id immediately.

Fast, non-blocking volume lifecycle with a durable audit of every change.

How · REST `POST /api/atlas/v1/volumes (kind: "block") → 202 + job_id`; `GET /volumes/{id}`, `DELETE /volumes/{id}`. CLI: `atlasctl create-volume NAME --size-gib 5 --policy database`. Console: Storage Center → Volumes → Create.



CephFS File Shares (RWX)

Provision shared read-write-many file storage on CephFS for workloads that need concurrent access.

Multi-writer file storage without standing up a separate NAS.

How · REST `POST /api/atlas/v1/volumes` with a CephFS/RWX placement (e.g. `kubernetes.storage_class: "zyvor-cephfs-shared"`, `access_mode: ReadWriteMany`). Console: Storage Center → Volumes → Create (File).



Object Buckets

Create S3 buckets via ObjectBucketClaim with per-bucket quotas, stats, and presigned upload/download URLs.

Self-service object storage with quota control and short-lived access links.

How · REST `POST /api/atlas/v1/buckets` with `{ "name": "...", "max_objects": ..., "max_size": "2G" }`; list `GET /buckets`. CLI: `atlasctl create-bucket NAME`. Console: Storage Center → Buckets.



Direct RBD Image Ops

Provision, clone, resize, flatten, snapshot, and roll back RBD images directly, with per-image usage tracking.

Full low-level control when you need to bypass the PVC abstraction.

How · REST under `POST /api/atlas/v1/rbd-images/{pool}/{image}/...` — `/resize`, `/qos`, `/migrate`. Console: Storage Center → Ceph → RBD Images.



Safe Resize (Grow & Shrink)

Expand volumes freely; shrink is opt-in behind an explicit `allow_shrink` flag as a data-loss guard.

Reclaim over-provisioned space without accidentally destroying data.

How · Grow: `POST /api/atlas/v1/volumes/{id}/expand { "new_size_bytes": ... }`. Shrink: `POST /api/atlas/v1/rbd-images/{pool}/{image}/resize { "size_bytes": ..., "allow_shrink": true }`.



Kubernetes Storage Inventory

Read-only listing of StorageClasses, PVCs, and PVs live from the cluster via kube-rs.

See exactly how Atlas storage surfaces inside Kubernetes.

How · REST `GET /api/atlas/v1/storage-classes · /kubernetes/pvcs · /kubernetes/pvs` (needs a reachable cluster; `502` otherwise). CLI: `atlasctl storage-classes`.

Data Protection

Snapshots, clones, and off-cluster backups — scheduled, verified, and retention-managed.



Snapshots

Take point-in-time volume snapshots, then clone or restore from them behind a safe-delete guard.

Instant rollback points without copying whole volumes.

How · REST POST
`/api/atlas/v1/volumes/{id}/snapshots ; list GET /snapshots ; DELETE /snapshots/{id} (409 if cloned from, ?force=true overrides). CLI: atlasctl snapshot-volume VOLUME_ID.`



Clone & Restore

Spin a new volume from a snapshot or restore a volume in place from any snapshot.

Branch environments from a known-good state in seconds.

How · REST POST
`/api/atlas/v1/snapshots/{id}/clone` (new independent volume) or `.../restore` (point-in-time copy). CLI: `atlasctl clone-snapshot SNAPSHOT_ID --name NAME`.



Off-Cluster Backups

Stream RBD export-diff to RGW/S3 as verified multipart uploads, with restore-from-data and presigned downloads.

Durable, verifiable backups that survive loss of the source pool.

How · REST POST `/api/atlas/v1/backup-jobs { "volume_id": ..., "bucket_id": ..., "mode": "data" } ; restore via POST /restore-jobs ; presigned download GET /backups/{id}/download?what=data`. CLI: `atlasctl backup-volume VOLUME_ID --bucket-id BUCKET_ID`.



Scheduled Snapshots & Backups

Register recurring snapshot and backup schedules per volume, driven by background workers.

Set-and-forget protection instead of manual, forgettable runs.

How · REST POST
`/api/atlas/v1/volumes/{id}/schedule` with `{ "interval_secs": 3600, "keep": 24 }` (or `{ "kind": "backup", "bucket_id": ..., "mode": "data" }`); list `GET /schedules`.



Retention Policies

Keep-N plus max-age retention automatically prunes old backups (and object versions).

Bounded storage cost without hand-pruning stale copies.

How · Set `keep` (keep-N) and `max_age_secs` on `POST /api/atlas/v1/backup-jobs` or on a schedule; defaults come from `ATLAS_BACKUP_KEEP` / `ATLAS_BACKUP_MAX_AGE_SECS`. Pruning runs as delete jobs after each backup.



Orphan Backup GC

Surface backups whose source volume is gone so they can be cleaned up deliberately.

No silent buildup of orphaned data you're still paying to store.

How · REST GET
`/api/atlas/v1/maintenance/orphans` lists dangling backups; clean each with `DELETE /api/atlas/v1/backups/{id}`.

Multi-Backend & Ceph Operations

Run Ceph, NFS, and ZFS side by side — and operate Ceph natively when it's the driver.



Three Storage Backends

Ceph, NFS, and ZFS all live behind the same StorageDriver, with filters, per-backend gauges, and a backends summary.

Mix backends under one control plane and one API.

How · REST GET /api/atlas/v1/backends ; filter inventory with GET /api/atlas/v1/pools?backend=<id> and /volumes?backend=<id> . Console: Storage Center → Backends.



Dynamic Backend Registration

Register a live NFS or ZFS backend over the API and Atlas instantiates the driver and discovers it immediately.

Onboard new storage without redeploying the gateway.

How · REST POST /api/atlas/v1/backends { "backend_type": "nfs"|"zfs", "server": ..., "targets": [...] } — instantiates the driver and discovers on the spot. Console: Storage Center → Backends → Add.



Backend Cordon & Drain

Cordon a backend to reject new provisioning (503) while you drain and service it, then uncordon.

Take storage offline for maintenance without breaking callers.

How · REST POST /api/atlas/v1/backends/{id}/cordon then .../uncordon (creates against a cordoned backend return 503). Console: Storage Center → Backends → Cordon.



Ceph-Native Introspection

Read ceph status, osd-tree, osd-df, and df directly through dedicated endpoints.

Diagnose the real cluster without leaving the control plane.

How · REST GET /api/atlas/v1/ceph/status · /ceph/osd-tree · /ceph/osd-df · /ceph/df . CLI: atlasctl ceph-status | ceph-osd-tree | ceph-osd-df | ceph-df .



OSD Management

Mark OSDs in or out and reweight them via ceph osd commands from the API.

Rebalance and service OSDs through the same audited surface.

How · REST POST /api/atlas/v1/osds/{osd_id}/out · /in · /reweight?weight=0.8 → 202 + job_id (real Ceph).



QoS & Pool Migration

Set per-image IOPS/BPS QoS limits and migrate images between pools with rbd migration.

Tune performance and rebalance placement without downtime.

How · REST POST /api/atlas/v1/rbd-images/{pool}/{image}/qos?iops=&bps= (0 clears a cap) and .../migrate?dest_pool=<pool> → 202 + job_id (real Ceph).

OSD, QoS, and pool-migration operations act on a **real Ceph cluster** — they are exercised fake-first in CI and fully verified against live Rook Ceph.

Observability & Metrics

Live capacity, health, forecasts, and a cinematic console view of the whole estate.



Prometheus Metrics

Self-metrics at `/metrics` plus Ceph metrics and per-backend gauges for scraping.

[Drop Atlas straight into your existing monitoring stack.](#)

How · REST `GET /metrics` (Prometheus text exposition) and `GET /api/atlas/v1/metrics/ceph[?prefix=ceph_osd]`. CLI: `atlasctl self-metrics / atlasctl ceph-metrics`.



Persisted Time-Series History

Capacity and usage history is persisted and served at `/metrics/history` for trend analysis.

[See where storage has been, not just where it is now.](#)

How · REST `GET /api/atlas/v1/metrics/history`. CLI: `atlasctl history --minutes 60`.



Capacity Forecast

A days-to-full forecast projects when pools and backends will run out of headroom.

[Buy or reclaim capacity before you hit the wall.](#)

How · REST `GET /api/atlas/v1/metrics/forecast`. CLI: `atlasctl forecast --minutes 1440`.



Observatory

Six live canvas visualizations render the cluster, pools, jobs, and flows in real time in the console.

[An at-a-glance operations view that reads like mission control.](#)

How · Console only: Storage Center → Observatory (live canvas views fed by the metrics and `/events` streams).



Unified Activity Feed

A single `/events` stream merges activity across every backend and subsystem.

[One timeline for everything happening in storage.](#)

How · REST `GET /api/atlas/v1/events` (SSE stream). Console: Storage Center → Activity.



Grafana Bundle

A ready-to-apply Prometheus + Grafana observability bundle ships in `deploy/observability`.

[Stand up dashboards in minutes instead of building them from scratch.](#)

How · Apply the shipped bundle: `kubectl apply -f deploy/observability` (Prometheus scrapes `/metrics`; import the Grafana dashboards).

SECTION 6

Alerting & Day-2 Ops

Atlas is operable, not just observe-and-provision — with alerts, maintenance, and safe upgrades.



Rule-Based Alerting

Rules cover cluster health, pool near-full, OSD down/latency, capacity forecast, recovery, failing jobs, CDC errors, and tenant-quota thresholds.

Catch storage problems by policy, not by luck.

How · REST GET `/api/atlas/v1/alerts[?state=open]` ; run rules on demand with POST `/api/atlas/v1/alerts/evaluate` (also every `ATLAS_MONITOR_INTERVAL_SECS`).
Console: Storage Center → Alerts.



Alert Lifecycle

Acknowledge, silence for a window, or resolve alerts, with a single webhook sink for downstream notification.

Route real signal to your team and mute the noise.

How · REST POST `/api/atlas/v1/alerts/{id}/ack` · `/silence[?secs=3600]` · `/resolve` ; webhook sink via `ATLAS_ALERT_WEBHOOK_URL` .



Maintenance Pause

Pause the job worker so new work holds in queue until you resume — cluster-wide freeze for maintenance.

Do risky work with nothing new landing mid-flight.

How · REST POST `/api/atlas/v1/maintenance { "paused": true }` (resume with `false`); read state with GET `/api/atlas/v1/maintenance` .



Upgrade Pre-Flight

A preflight check blocks upgrades on HEALTH_ERR clusters, open critical alerts, in-flight jobs, or lagging CDC.

Never ship an upgrade into an unhealthy cluster.

How · REST GET `/api/atlas/v1/upgrade/preflight` → `{ ready, checks, blockers }` ; `scripts/deploy-remote.sh` gates on it automatically.



Gated Rollout & Rollback

deploy-remote.sh auto-gates on pre-flight and supports rollout undo, with a --force override.

[Ship and un-ship with guardrails baked into the script.](#)

How · `scripts/deploy-remote.sh`
`<host> <user>` (auto-gates on pre-flight + rollout-restarts);
`scripts/deploy-remote.sh`
`<host> <user> --rollback` for
`kubectl rollout undo; --force` to override.



Control-Plane State Backup

Optional VACUUM INTO snapshots of the control-plane DB stream to S3/RGW on a schedule with keep-N retention.

[Recover the control plane itself, not just the data it manages.](#)

How · Enable via env: `ATLAS_STATE_BACKUP_SECS` +
`ATLAS_STATE_BACKUP_ENDPOINT / _BUCKET / _ACCESS_KEY / _SECRET_KEY`
(`_KEEP` default 24). Off by default.

Governance & Multi-Tenancy

Tenant isolation, quotas, policy, audit, and cost attribution — built into the control plane.



Multi-Tenancy

Per-tenant quotas and policy overrides isolate consumers of shared storage.

Give each team its own guardrails on one physical cluster.

How · REST GET|PUT
`/api/atlas/v1/tenants/{id}/quota` and `GET /tenants/{id}/policies` · PUT|DELETE
`/tenants/{id}/policies/{intent}` . Console:
 Storage Center → Tenants.



Tenant Quotas

Set and read capacity quotas per tenant, with alerts as usage approaches 80% and 95%.

Prevent one tenant from starving everyone else.

How · REST PUT
`/api/atlas/v1/tenants/{id}/quota { "max_bytes": ..., "max_volumes": ... }` (admin); GET returns limits + live usage. A create that would exceed a limit is rejected `409` before enqueue.



Placement Policies & Drift

Intent-to-placement policies govern where storage lands, and a policy-drift report flags volumes that no longer match.

Keep placement compliant with intent over time, not just at creation.

How · REST GET `/api/atlas/v1/policies` (catalog), PUT `/tenants/{id}/policies/{intent}` (per-tenant override), and GET `/api/atlas/v1/policy-drift` (volumes off their policy).



Service-Account JWTs & RBAC

Issue role-scoped JWTs and revoke leaked tokens before their TTL, enforced by RBAC across REST and gRPC.

Grant least-privilege access and kill compromised tokens instantly.

How · REST POST `/api/atlas/v1/auth/tokens { "subject": ..., "role": "operator", "ttl_secs": 3600 }` (admin); revoke with POST `/auth/tokens/{jti}/revoke`, list GET `/auth/tokens/revoked` .



Audit Log

Every mutating action is recorded, exportable as CSV, with configurable retention-day pruning.

A defensible trail of who changed what, when.

How · REST GET `/api/atlas/v1/audit.csv` (SIEM export); retention via `ATLAS_AUDIT_RETENTION_DAYS` (0 = keep forever).



Chargeback

Attribute cost per tenant from a configurable USD-per-GiB-month rate.

Turn shared storage into a billable, accountable service.

How · REST GET `/api/atlas/v1/chargeback` (per-tenant usage + optional cost); set the rate with `ATLAS_CHARGEBACK_USD_PER_GIB_MONTH` .

Governance controls include a per-minute **rate limiter** (429 on breach) so a runaway caller can't overwhelm the gateway.

DataBridge — Database Mobility

Migrate managed cloud databases to open, self-managed engines on Ceph at the edge.

Source engine	Edge target	Migration type	Real connector
PostgreSQL	CloudNativePG	homogeneous	default (tokio-postgres)
MySQL / MariaDB	Percona XtraDB	homogeneous	default (sqlx)
Oracle	CloudNativePG (Postgres)	heterogeneous	feature: oracle (OCI)
SQL Server	CloudNativePG (Postgres)	heterogeneous	feature: sqlserver
MongoDB	Percona Server for MongoDB	homogeneous (document)	feature: mongodb



Discover & Assess

Introspect a live source database (tables, PKs, sizes, WAL/CDC capability) and score edge-readiness.

Know exactly what you're migrating — and its risks — before you start.

How · REST POST

`/api/atlas/v1/databridge/sources` (register) → `.../sources/{id}/discover` ; create a plan and score it with `POST /databridge/plans/{id}/assess` (each returns a job).



Edge Provisioning on Ceph

Provision CloudNativePG, Percona XtraDB, or Percona Server for MongoDB with data and WAL on Ceph RBD.

Land your database on open engines and your own storage, off the cloud bill.

How · REST POST

`/api/atlas/v1/databridge/plans/{id}/provision` → `202 + job_id` ; read the result at `GET /databridge/edge-clusters` .



Full-Load + CDC Replication

Dump-and-restore full load, then Debezium streams ongoing changes (WAL/binlog/redo/oplog) via Kafka to the edge.

Migrate with the source still live — no big-bang downtime.

How · REST POST

`/api/atlas/v1/databridge/plans/{id}/full-load` , then `.../cdc/start` (`.../cdc/stop` to halt); watch lag via `GET /databridge/cdc-streams` .



Guarded Cutover & Rollback

Cutover is admin-guarded on validation-passed and low CDC lag, with rollback allowed inside a defined window.

Switch over safely and back out if something looks wrong.

How · REST POST

`/api/atlas/v1/databridge/plans/{id}/validate` → `.../cutover` (admin, guarded on validation + low CDC lag) → `.../rollback` (admin, within the rollback window).



Six Source Engines

PostgreSQL, MySQL, MariaDB, Oracle, SQL Server, and MongoDB — homogeneous and heterogeneous (to Postgres) paths.

[One migration control plane spanning your whole database estate.](#)

How · REST `POST`

```
/api/atlas/v1/databridge/sources { "kind":  
  "postgres"|"mysql"|... , "driver_mode":  
  "fake"|"real" } ; Oracle/SQL Server/MongoDB  
connectors are behind cargo features  
( oracle / sqlserver / mongodb ).
```



CDC Self-Heal

A stalled stream is re-established via `cdc/restart`, and the reconciler auto-restarts an unhealthy stream up to 3× before alerting.

[Replication recovers itself instead of quietly falling behind.](#)

How · REST `POST`

```
/api/atlas/v1/databridge/plans/{id}/cdc/restart  
re-establishes a stalled stream; the reconciler also  
auto-restarts up to 3× before raising a CDC  
replication error alert.
```

DataBridge — Object Migration

Move AI datasets, model weights, and RAG documents from cloud object stores into Ceph RGW.



Cloud-to-Ceph Object Copy

Copy objects from a cloud store into a Ceph RGW bucket so products consume from a local S3 endpoint.

Pull AI-scale datasets on-prem and off the cloud egress meter.

How · REST `POST /api/atlas/v1/databridge/object` (create the migration) then `POST /databridge/object/{id}/start` → `202 + job_id`; watch via `/jobs/{id}/watch`.



Incremental, Verified Copy

List source, diff against destination by key+size, stream each changed object recording sha256, then verify every key landed.

Trustworthy syncs that only move what changed and prove they arrived.

How · Automatic within the copy job; poll `GET /api/atlas/v1/databridge/object/{id}` for `objects_{total,done}` / `bytes_{total,done}` / `verified`.



Multi-Cloud Sources

AWS S3, GCS (S3-interop), and any S3-compatible store work today; Azure Blob has a native feature-gated connector.

One mover for the object stores you actually use.

How · Set the source endpoint/type in the `POST /api/atlas/v1/databridge/object` body; Azure Blob is behind the `azure-blob` cargo feature.



Concurrent Streaming

Objects stream and copy concurrently with configurable part size and concurrency for AI-scale datasets.

Move terabytes fast instead of one slow object at a time.

How · Tune part size and concurrency in the migration config on `POST /api/atlas/v1/databridge/object`; the copy runs as an async job.



Secret-Ref Credentials

Credentials are referenced from a Kubernetes Secret, resolved in-process at run time, and never stored or logged.

Migrate data without spilling access keys into the control plane.

How · Pass a Kubernetes Secret reference (not raw keys) in the migration body; Atlas resolves it in-process at run time and never persists or logs the keys.

Disaster Recovery

Cross-cluster RBD mirroring and failover scaffolding for a second Ceph site.



Cluster Peering

Register and remove peer Ceph clusters as DR targets through the control plane.

[Catalog your DR relationships in one place.](#)

How · REST POST `/api/atlas/v1/dr/peers`
(bootstrap token via a k8s Secret ref) · GET
`/dr/peers` .



Volume Mirroring

Enable or disable rbd mirror on a volume and list all mirrors with their DR status.

[Keep critical volumes replicated to a second site.](#)

How · REST POST
`/api/atlas/v1/volumes/{id}/mirror?`
`mode=snapshot&peer=<id>` · DELETE `.../mirror` ; list
GET `/dr/mirrors` and posture GET `/dr/status` .



Promote & Demote (Failover)

Promote a mirror at the DR site or demote the primary to orchestrate failover.

[Flip sites when the primary goes down.](#)

How · REST POST
`/api/atlas/v1/dr/mirrors/{id}/promote`
(secondary → primary) · `/demote` (the reverse).

Honest status: the DR **control-plane catalog, API, and jobs are complete**, but the underlying **rbd mirror operations are not yet verified against a live second Ceph cluster** — treat DR as scaffolding pending a two-site validation.

Storage Center Console

A Zeus OS-style React console embedded in the gateway, wiring every capability to a UI.



Embedded React Console

A React/Vite/Tailwind SPA served from the gateway binary over HTTPS with a branded login.

Nothing extra to deploy — the UI ships inside Atlas.

How · Browse to `http://127.0.0.1:5110/` (local) or `http://<node>:30511/` (Ceph-mode NodePort) — the SPA is served from the same gateway binary.



Full-Capability Coverage

Inventory, capacity/health, job progress via SSE, alerts, metrics, tenants, and every write action are wired into the UI.

Drive the whole control plane without touching the CLI.

How · Storage Center left-nav — Inventory, Capacity, Jobs, Alerts, Tenants, Backends — each backed by the matching `/api/atlas/v1/*` endpoints.



Live Job Progress (SSE)

Async jobs stream progress to the console over server-sent events as they run.

Watch provisioning, backups, and migrations advance in real time.

How · Console Jobs view subscribes to `GET /api/atlas/v1/jobs/{id}/watch (text/event-stream)`; the gRPC parity is `WatchJob`.



Themeable Design

A Nebula default theme plus Midnight and Aurora variants in the Zeus OS "Tahoe" design language.

A console that looks like the rest of the Zyor suite.

How · Switch themes (Nebula / Midnight / Aurora) from the Storage Center appearance settings.



atlasctl CLI

A headless REST client covering health, discovery, volumes, snapshots, backups, buckets, RBD, tenants, tokens, and more.

Script and automate everything the console can do.

How · `cargo run -p atlas-cli -- <command>` (e.g. `atlasctl health`, `atlasctl volumes`, `atlasctl create-volume ...`); global flags `--base-url` (`ATLAS_BASE_URL`) and `--token` (`ATLAS_TOKEN`).

Support & next steps

Support & contact

Atlas is developed by **ZyvorAI Labs**. For onboarding, a proof-of-concept, or a guided demo, contact your account team at **info@zyvor.dev**.

Good to know: Atlas is at slices 1–5 plus DataBridge, verified end-to-end on real k3s + Rook Ceph clusters. Some capabilities require real infrastructure or optional build features: cross-cluster DR (rbd mirror) is complete as control-plane scaffolding but unverified without a live second Ceph cluster; true multi-replica HA requires Postgres (single-replica SQLite is the default); Oracle, SQL Server, MongoDB, precise CDC lag, and Azure Blob connectors are behind cargo features that link native dependencies; and OSD/QoS/pool-migration and live CDC for non-Postgres engines exercise real Ceph and a running Kafka/Debezium stack respectively.

Atlas

One intent-based API that maps every product's storage request to Ceph, NFS, ZFS, or the cloud — while owning inventory, ownership, protection, and audit.

info@zyvor.dev · zyvor.dev